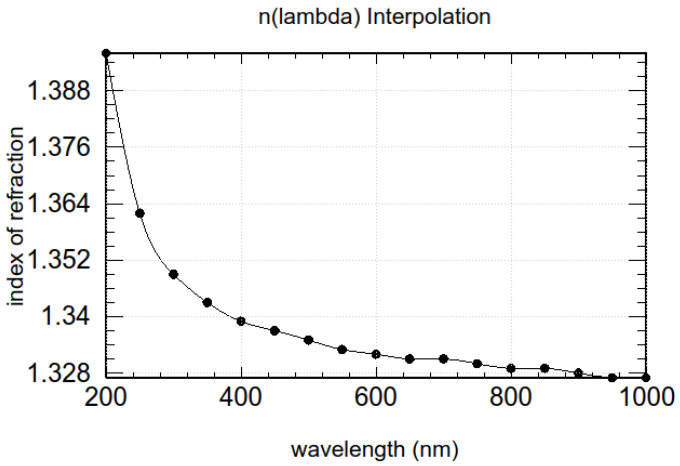
Computational Methods #5

CH4 EX7

To implement a cubic spline interpolation we construct an instance of the `InterpolatingFunction` class.

```
Genfun::InterpolatingFunction interpolation(Genfun::CUBIC_SPLINE);
```

Using the `addPoint(double x, double y)` method add all the points in the table and generate a plot.

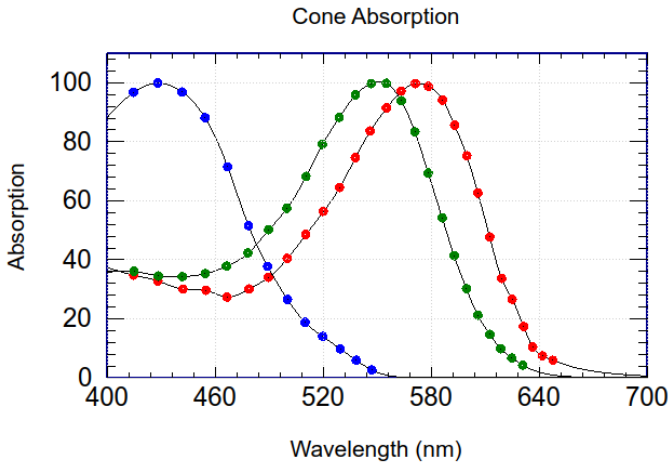


CH4 EX8

First task is to generate csv files from the image which I did using the online tool `plotdigitizer.com`. The second task is to parse the csv files and dump the data into `std::vector<std::vector<double>>` objects which is accomplished by opening the file as an `std::ifstream` object and parsing through the lines and commas with `while` loops.

The interpolation functions are generated the same as in the last exercise but for the sake of the next exercise I first perform an extrapolation procedure to extend all the data to $\lambda = 700$ nm otherwise

there is no way to use a cubic spline to fit the curves as it will extend a straight line to $-\infty$ instead of exponentially decaying to zero. So, using the value and the derivative of each curve at the end of its data set I stitch it with a decaying exponential such that the function and derivative are contiuous.



CH4 EX9

Let the absorbance the cones be $R(\lambda), G(\lambda), B(\lambda)$. The goal is to generate a transformation from these absorbance functions to r, g, b which are used to generate an RGB color. There are standard transformations for this, I implemented a naive transformation where I scale the values $R(\lambda), G(\lambda), B(\lambda)$ at each wavelength by a factor $\alpha(\lambda)$ such that $\alpha(R+G+B) = 255$. Then use these scaled quantities $r = \alpha R, g = \alpha G, b = \alpha B$ to generate the image.

To generate the image create an instance of the `QImage` class. Then for set the color of each pixel to the correct r, g, b values using a for loop.

```
  QImage image(800, 600, QImage::Format_RGB32);
QRgb rgbColor;
double wavelength{0};

for(int i=0; i<800;i++){
  wavelength = 400.0 + (i * 300.0 / 800.0);

  double alpha = 255.0/(interpolationRed(wavelength)
                      + interpolationGreen(wavelength)
                      + interpolationBlue(wavelength));
  double r = alpha * interpolationRed(wavelength);
  double g = alpha * interpolationGreen(wavelength);
  double b = alpha * interpolationBlue(wavelength);

  for(int j=0; j<600; j++){
    image.setPixel(i, j, qRgb(r, g, b));
  }
}
```



The largest problem with this naive transform is the fact that the red and green curves have alot of overlap so the next best transform would be another linear transformation that differentiates between green and red and green better.

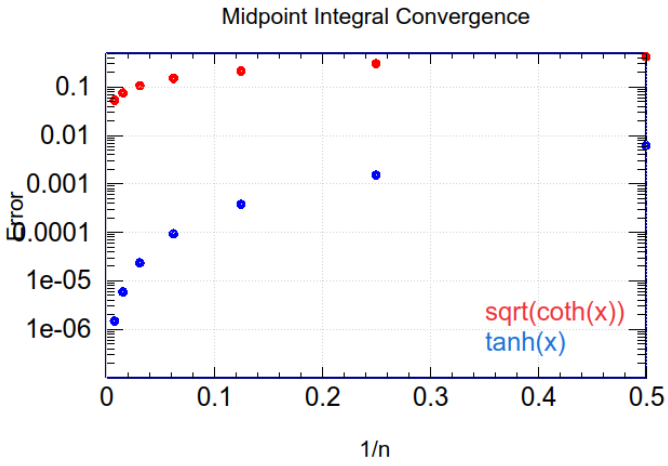
CH5 EX1

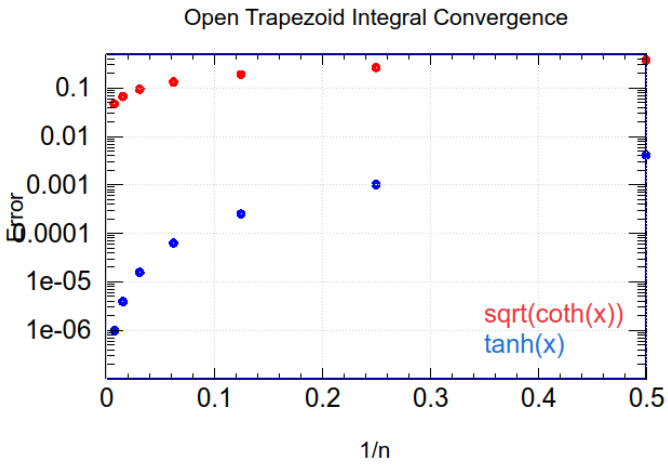
To integrate a `Genfun::AbsFunction` object use the `Genfun::SimpleIntegrator` class. To choose the method for the integration use the `Genfun::QuadratureRule`. The program `integrate` is build such that we can choose the integration method from the command line. This was implemented using smart pointers in the function `getRule(std::string name)`. Then the rule is called into the `main` program by dereferencing the pointer.

```
std::unique_ptr<Genfun::QuadratureRule> getRule(std::string name) {
  std::transform(name.begin(), name.end(), name.begin(), ::tolower);

  if (name == "trapezoid"){
        return std::make_unique<Genfun::TrapezoidRule>();}
  if (name == "opentrapezoid"){
        return std::make_unique<Genfun::OpenTrapezoidRule>();}
  if (name == "midpoint"){
        return std::make_unique<Genfun::MidpointRule>();}
  if (name == "simpson"){
        return std::make_unique<Genfun::SimpsonsRule>();}

  std::cerr << "Unknown rule \"" << name << "\".\n"
            << "Valid options:
            << trapezoid, opentrapezoid, midpoint, simpson\n";
  return nullptr;
}
```





The function $\sqrt{\coth(x)}$ diverges at $x = 1$. This implies we need to use open quadrature rules to avoid evaluating the function at the endpoints, this is not a problem for the midpoint rule but for the trapezoid rule we need to use an `Genfun::OpenTrapezoid()` method and for the Simpson rule there is no open implementation so only $\tanh(x)$ is implemented.

CH5 EX2

The total energy in the pendulum is given by

$$E = \frac{1}{2}mR^2\dot{\theta}^2 - mgR\cos\theta$$

From here we solve for $\dot{\theta}$

$$\dot{\theta}^2 = 2\left(E/mR^2 + g/R\cos\theta\right)$$

For small angles the period of a pendulum is given by $T_0 = 2\pi\sqrt{R/g}$. Writing $\dot{\theta}$ in terms of this time unit we get

$$\dot{\theta} = \frac{2\sqrt{2}\pi}{T_0}\sqrt{E/mgR + \cos\theta}$$

The total energy E in the oscillator can be written in terms of $\theta_{\max}$ by setting $\dot{\theta} \to 0$ and $\theta \to \theta_{\max}$

$$E = -mgR\cos\theta_{\max}$$

From here the final expression of $\dot{\theta}$ is given by

$$\dot{\theta} = \frac{2\sqrt{2}\pi}{T_0}\sqrt{\cos\theta - \cos\theta_{\max}}$$

Separating this equation and integrating for the period $T = T_0 f(\theta_{\max})$

$$f(\theta_{\max}) = \frac{1}{\pi\sqrt{2}}\int_{-\theta_{\max}}^{\theta_{\max}} \frac{d\theta}{\sqrt{\cos\theta - \cos\theta_{\max}}}$$

To compute this integral numerically first we implement the function $f(\theta_{\max})$

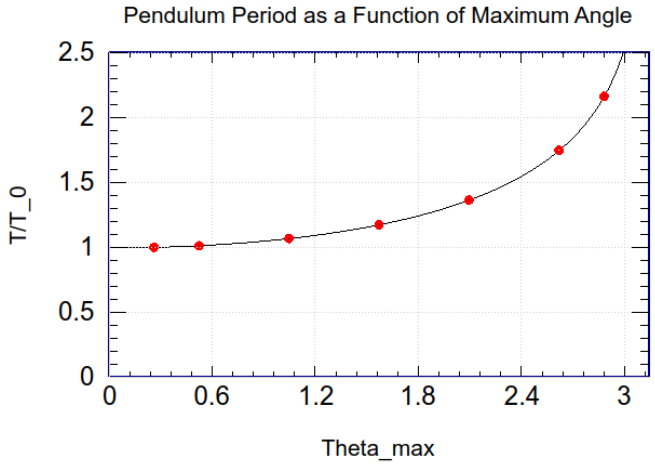
```
double f(double thetMax) {
  Genfun::GENFUNCTION cos = Genfun::Cos();
  Genfun::GENFUNCTION sqrt = Genfun::Sqrt();

  Genfun::GENFUNCTION integrand = 1/sqrt(cos-cos(thetaMax));

  Genfun::SimpleIntegrator integrator(0.0,thetaMax,Genfun::MidpointRule(),6);
  double result=sqrt(2.0)*integrator(integrand)/M_PI;
  return result;

}
```

From here we simply adapt the function with a `Genfun::F1D` adapter and plot it as usual.



As expected the period for small $\theta_{\max}$ is flat and has a value of T_0 indicating that this is valud for small angles. I had to use 1000 evaluatino points to in the integrator to get the initial value close enough to $T/T_0 = 1$.